



Local Search

Artificial Intelligence

Classical Search

- Designed to **explore search spaces systematically**
 - Systematicity is achieved by keeping one or more paths in memory and by recording which alternatives have been explored at each point along the path
 - When a goal is found, **the path to that goal constitutes a solution** to the problem
- Observable, deterministic, known environments where the solution is a sequence of actions

Local Search

- Evaluates and modifies one or more current states rather than systematically exploring paths from an initial state
 - Not systematic
- Suitable for solving
 - **problems in which all that matters is the solution state**, not the path cost to reach it, e.g., 8 Queen problem
 - **pure optimization problems**, in which the aim is to find the best state according to an objective, the path to the goal does not matter
 - e.g., nature provides an objective function—reproductive fitness—that Darwinian evolution could be seen as attempting to optimize, but there is no “goal test” and no “path cost” for this problem

Local Search

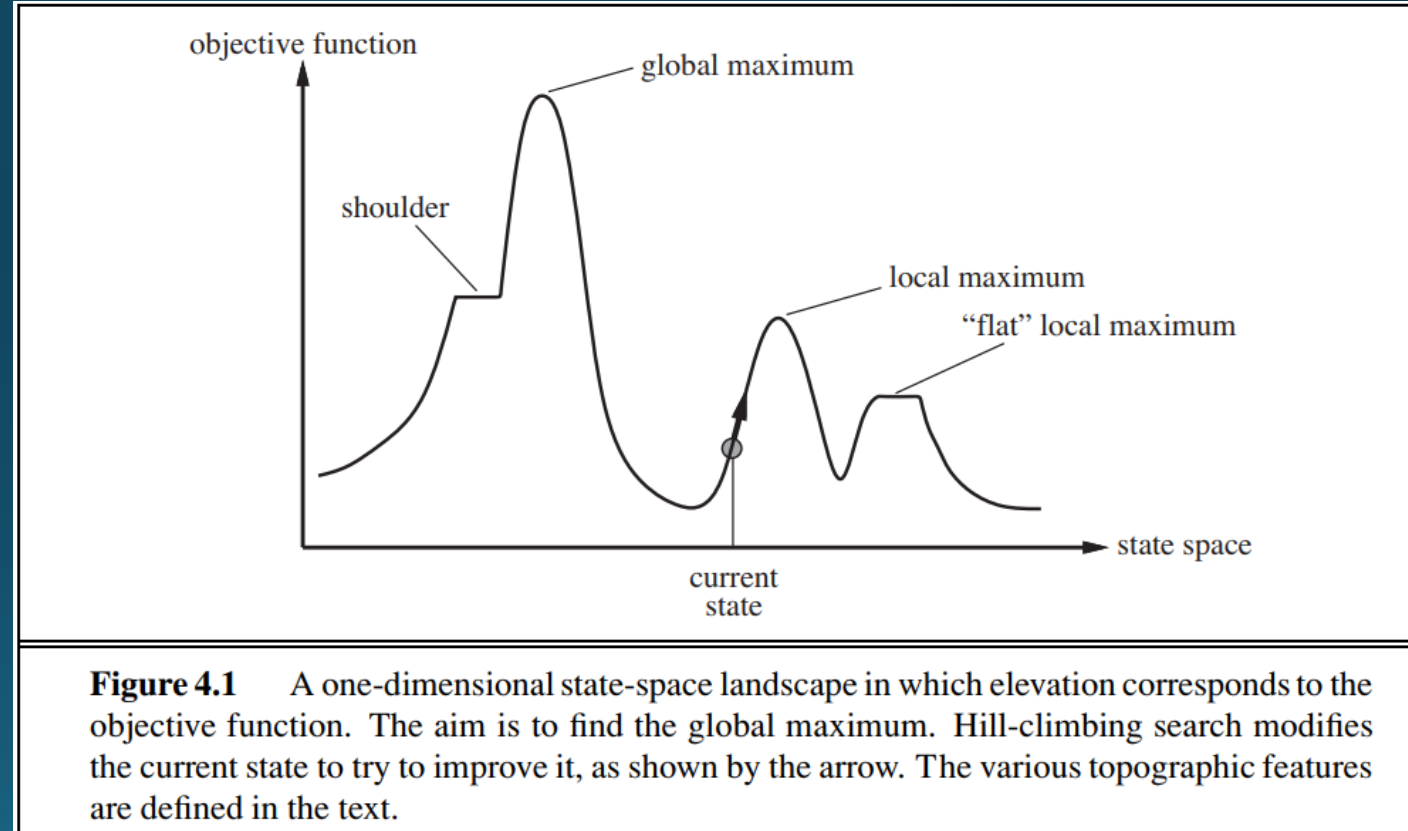
- Operates using a single current node (rather than multiple paths) and generally move only to neighbors of that node. Typically, the paths followed by the search are not retained.
- **Advantages**
 - use very little memory—usually a constant amount
 - can often find reasonable solutions in large or infinite (continuous) state spaces for which systematic algorithms are unsuitable

Local Search Algorithms

- Hill-climbing
- Simulated annealing (inspired by statistical physics)
- Beam Search
- Genetic algorithms (inspired by evolutionary biology)

State-space landscape

- Location - defined by the state
- Elevation – defined by the value of the heuristic cost function or objective function
 - If elevation corresponds to cost, then the aim is to find the lowest valley—a **global minimum**
 - if elevation corresponds to an objective function, then the aim is to find the highest peak—a **global maximum**
 - You can convert from one to the other just by inserting a minus sign.
- A complete local search algorithm always finds a goal if one exists
- An optimal algorithm always finds a global minimum/maximum



function HILL-CLIMBING(*problem*) **returns** a state that is a local maximum

current $\leftarrow$ MAKE-NODE(*problem*.INITIAL-STATE)

loop do

neighbor $\leftarrow$ a highest-valued successor of *current*

if *neighbor*.VALUE $\leq$ *current*.VALUE **then return** *current*.STATE

current $\leftarrow$ *neighbor*

Figure 4.2 The hill-climbing search algorithm, which is the most basic local search technique. At each step the current node is replaced by the best neighbor; in this version, that means the neighbor with the highest VALUE, but if a heuristic cost estimate h is used, we would find the neighbor with the lowest h .

Hill-climbing search

Hill-climbing search

Steepest-ascent version

```
function HILL-CLIMBING(problem) returns a state that is a local maximum  
  
  current  $\leftarrow$  MAKE-NODE(problem.INITIAL-STATE)  
  loop do  
    neighbor  $\leftarrow$  a highest-valued successor of current  
    if neighbor.VALUE  $\leq$  current.VALUE then return current.STATE  
    current  $\leftarrow$  neighbor
```

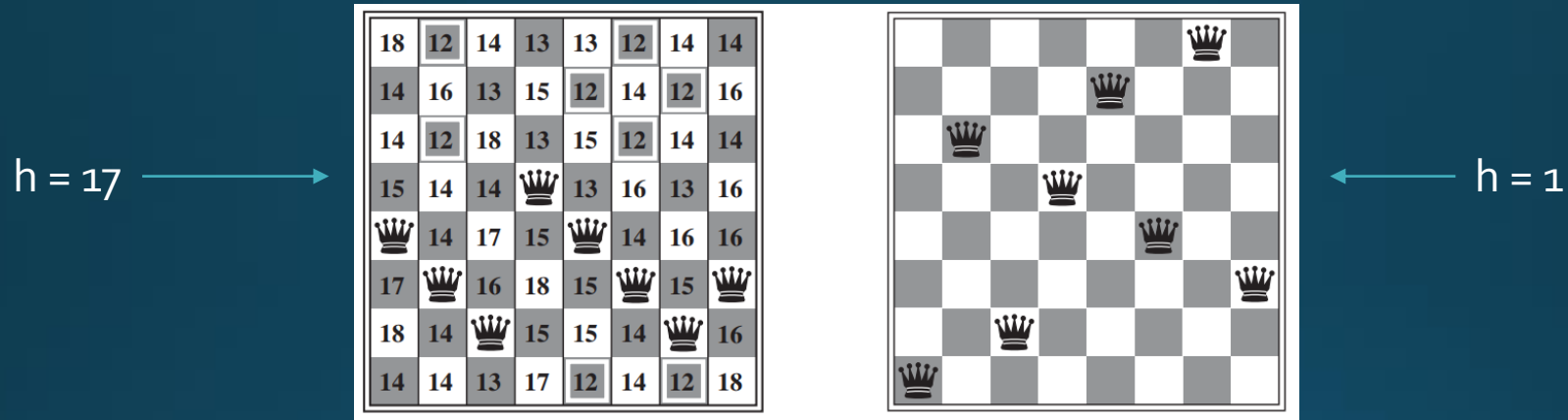
- It is simply a loop that continually moves in the direction of increasing value—that is, **uphill**
- It **terminates** when it reaches a **peak** where no neighbor has a higher value
- It does not maintain a search tree, so the data structure for the current node need only record the state and the value of the objective function

Hill-climbing search

- Does not look ahead beyond the immediate neighbors of the current state. This resembles trying to find the top of Mount Everest in a thick fog while suffering from amnesia.
- Sometimes called **greedy local search** because it grabs a good neighbor state without thinking ahead about where to go next

Hill-climbing search

Example: 8-queens problem



- Typically uses a complete-state formulation, where each state has 8 queens on the board, one per column
- The successors of a state are all possible states generated by moving a single queen to another square in the same column
 - each state has $8 \times 7 = 56$ successors
- The **heuristic cost function** h is the number of pairs of queens that are attacking each other, either directly or indirectly
- The global minimum of this function is zero, which occurs only at perfect solutions

Hill-climbing search

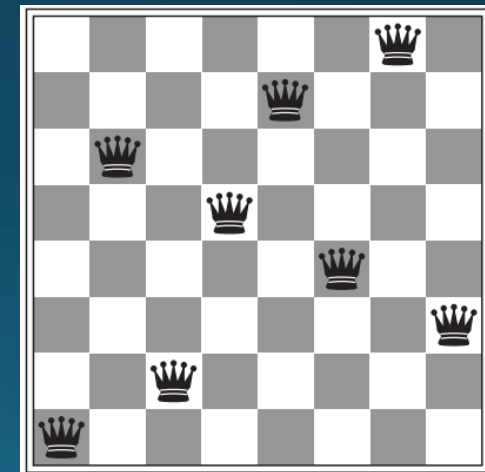
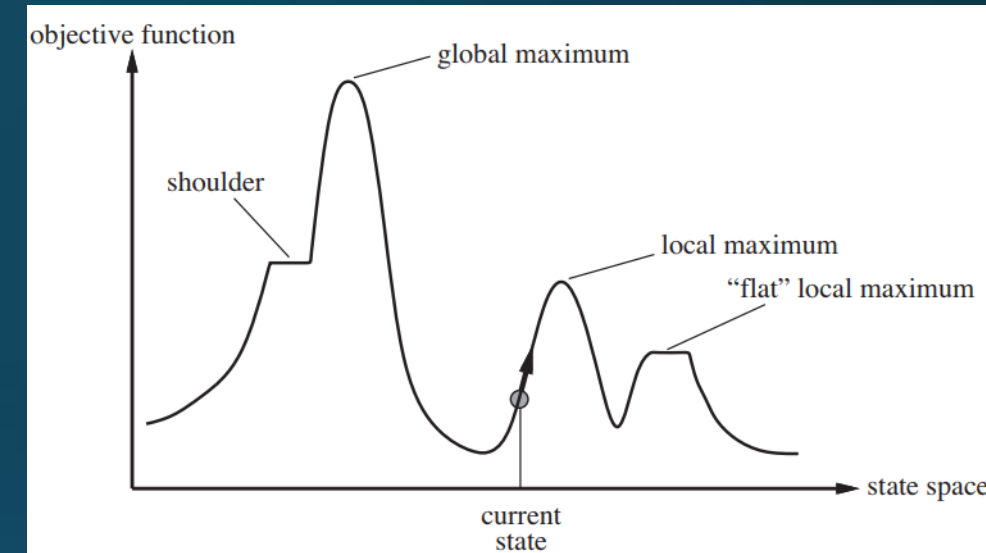
Steepest-ascent version

- Unfortunately, it often gets stuck for the following reasons:
 - Local maxima
 - Ridges
 - Plateaux
- In each case, the algorithm reaches a point at which no progress is being made

Hill-climbing search

Steepest-ascent version

- Unfortunately, it often gets stuck for the following reasons:
 - Local maxima** - a peak that is higher than each of its neighboring states but lower than the global maximum
 - Hill-climbing algorithms that reach the vicinity of a local maximum will be drawn upward toward the peak but will then be stuck with nowhere else to go
 - Ridges**
 - Plateaux**

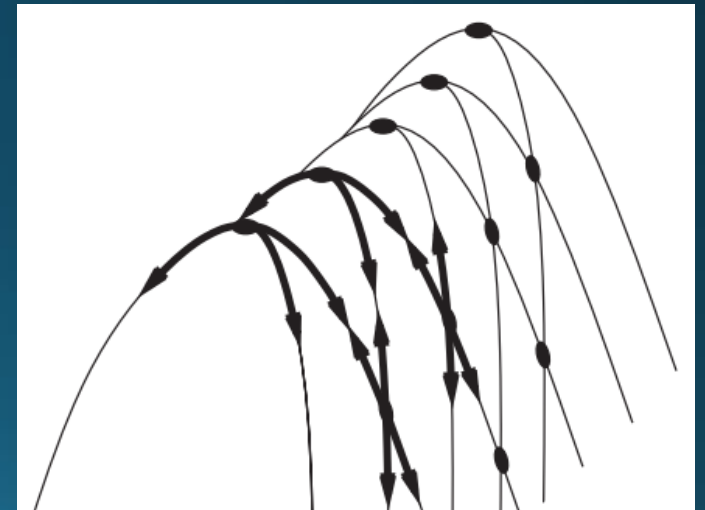


A local maxima; Every move of a single queen makes the situation worse

Hill-climbing search

Steepest-ascent version

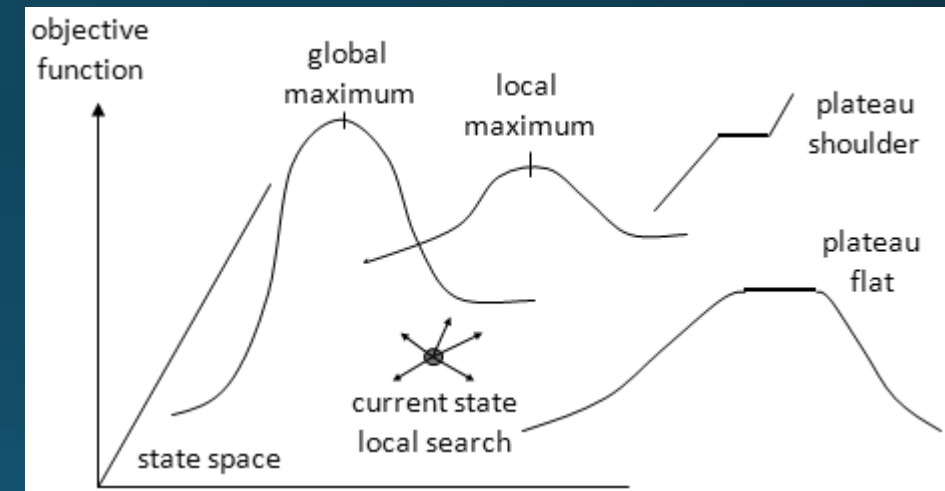
- Unfortunately, it often gets stuck for the following reasons:
 - Local maxima
 - Ridges
 - A ridge or a mountain ridge is a geographical feature consisting of a chain of mountains or hills that form a continuous elevated crest for some distance. The sides of the ridge slope away from narrow top on either side
 - Ridges result in a sequence of local maxima that is very difficult for greedy algorithms to navigate
 - Plateaux



Hill-climbing search

Steepest-ascent version

- Unfortunately, it often gets stuck for the following reasons:
 - Local maxima
 - Ridges
 - Plateaux - is a flat area of the state-space landscape
 - It can be a flat local maximum, from which no uphill exit exists, or a shoulder, from which progress is possible



Hill-climbing search

Steepest-ascent version

- Starting from a randomly generated 8-queens state, steepest-ascent hill climbing gets stuck 86% of the time, solving only 14% of problem instances.
 - It works quickly, taking just 4 steps on average when it succeeds and 3 when it gets stuck—not bad for a state space with $8^8 \approx 17$ million states
- Hill-climbing halts if it reaches a plateau where the best successor has the same value as the current state
 - How about allowing sideways moves in the hope that the plateau is really a shoulder?



Hill-climbing search

Steepest-ascent version

How about allowing sideways moves?

- Good, but be cautious
 - If we always allow sideways moves when there are no uphill moves, **an infinite loop will occur whenever the algorithm reaches a flat local maximum** that is not a shoulder.
 - One common solution is to **put a limit on the number of consecutive sideways moves** allowed.
- If we allow 100 consecutive sideways moves in the 8-queens problem, the success rate increases from 14% to **94%**.
 - Success comes at a cost: the algorithm averages roughly **21 steps for each successful instance** and 64 for each failure.

Hill-climbing search

Variants

- Steepest-ascent version (incomplete)
- Stochastic hill climbing (incomplete)
- First-choice hill climbing (incomplete)
- Random-restart hill climbing (trivially complete)

Hill-climbing variants

- Steepest-ascent version
- **Stochastic hill climbing**
 - Chooses at random from among the uphill moves; the probability of selection can vary with the steepness of the uphill move.
 - Usually converges more slowly than steepest ascent, but in some state landscapes, it finds better solutions.
- First-choice hill climbing
- Random-restart hill climbing

Hill-climbing variants

- Steepest-ascent version
- Stochastic hill climbing
- **First-choice hill climbing**
 - Implements stochastic hill climbing by generating successors randomly until one is generated that is better than the current state.
 - This is a good strategy when a state has many (e.g., thousands) of successors.
- Random-restart hill climbing

Hill-climbing variants

- Steepest-ascent version
- Stochastic hill climbing
- First-choice hill climbing
- **Random-restart hill climbing**
 - It conducts a series of hill-climbing searches from randomly generated initial states, until a goal is found
 - It is trivially complete with probability approaching 1, because it will eventually generate a goal state as the initial state
 - If each hill-climbing search has a probability p of success, then the expected number of restarts required is $1/p$
 - For 8-queens instances with no sideways moves allowed, $p \approx 0.14$
 - Expected number of iterations = $1/0.14 \approx 7$ (6 failures and 1 success).
 - The expected number of steps = $1 \times 4 + (1-p)/p \times 3 \approx 22$.
 - When we allow sideways moves, $1/0.94 \approx 1.06$ iterations are needed on average and $(1 \times 21) + (0.06/0.94) \times 64 \approx 25$ steps.
 - Even for three million queens, this approach can find solutions in under a minute

Hill-climbing

- The success of hill climbing depends very much on the shape of the state-space landscape:
 - if there are few local maxima and plateaux, random-restart hill climbing will find a good solution very quickly.
 - However, many real problems have many local maxima, ridges and plateaux
 - NP-hard problems typically have an exponential number of local maxima to get stuck on.
 - Despite this, a reasonably good local maximum can often be found after a small number of restarts

6 Queen problem using Hill climbing (steepest ascent)

Heuristic: #of queen pairs attacking each other

Action: randomly choose a row and move the queen in that row 1 cell

state.VALUE: -(#of queen pairs attacking each other)

X					
	X				
				X	
			X		
	X				
X					

function HILL-CLIMBING(*problem*) **returns** a state that is a local maximum

current $\leftarrow$ MAKE-NODE(*problem*.INITIAL-STATE)

loop do

neighbor $\leftarrow$ a highest-valued successor of *current*

if *neighbor*.VALUE $\leq$ *current*.VALUE **then return** *current*.STATE

current $\leftarrow$ *neighbor*

6 Queen problem using Hill climbing (steepest ascent)

Heuristic: #of queen pairs attacking each other

Action: randomly choose a row and move the queen in that row 1 cell

X					
	X				
			X		
			X		
	X				
X					

$$3+2+3+0+1+0=9$$

X					
	X				
				X	
			X		
	X				
X					

$$3+2+1+0+1+0=7$$

X					
	X				
					X
			X		
	X				
X					

$$3+2+0+0+1+0=6$$

	X				
	X				
					X
			X		
	X				
X					

$$2+2+0+0+1+0=4$$

23

function HILL-CLIMBING(*problem*) **returns** a state that is a local maximum

current $\leftarrow$ MAKE-NODE(*problem*.INITIAL-STATE)

loop do

neighbor $\leftarrow$ a highest-valued successor of *current*

if *neighbor*.VALUE $\leq$ *current*.VALUE **then return** *current*.STATE

current $\leftarrow$ *neighbor*